

Week-5

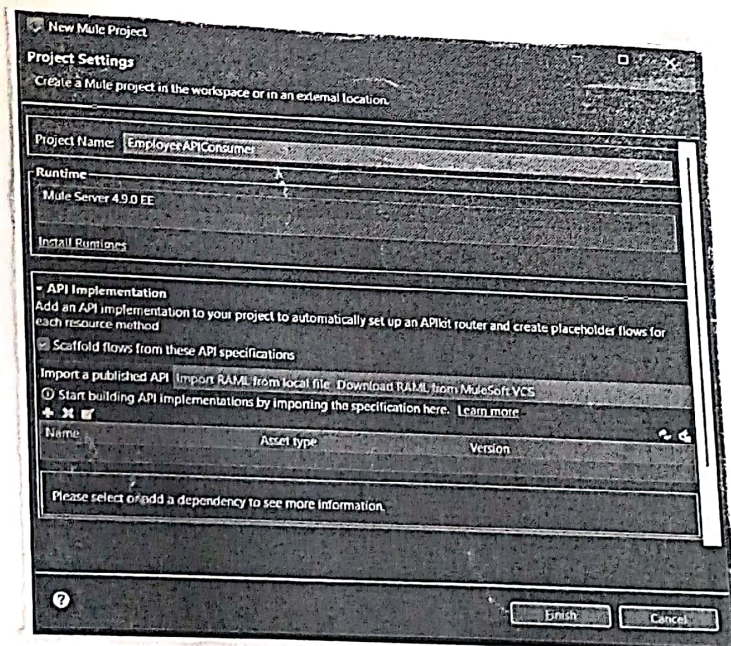
Create a mule application by importing the employee API from Anypoint platform into studio

Solution:

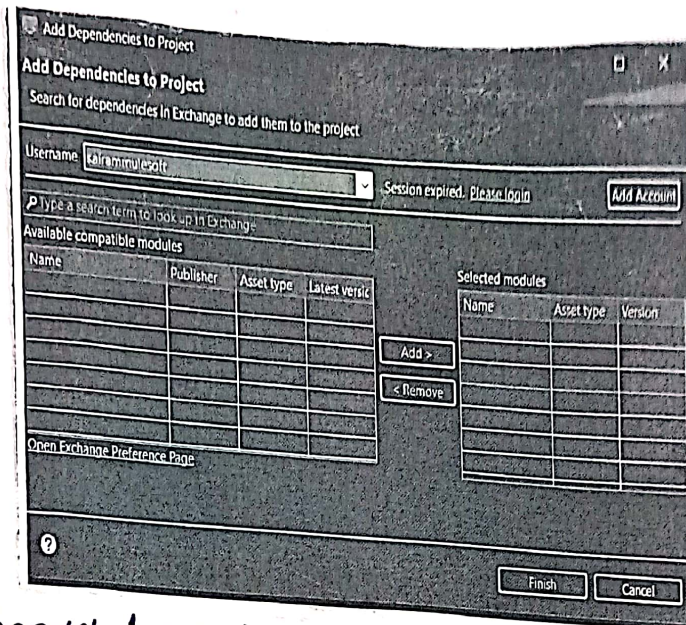
This experiment demonstrates how to import a designed and published RAML-based API from Anypoint Exchange into Anypoint studio. It also covers how to create a mule flow that sends a request to the mock endpoint and returns the response

Steps:

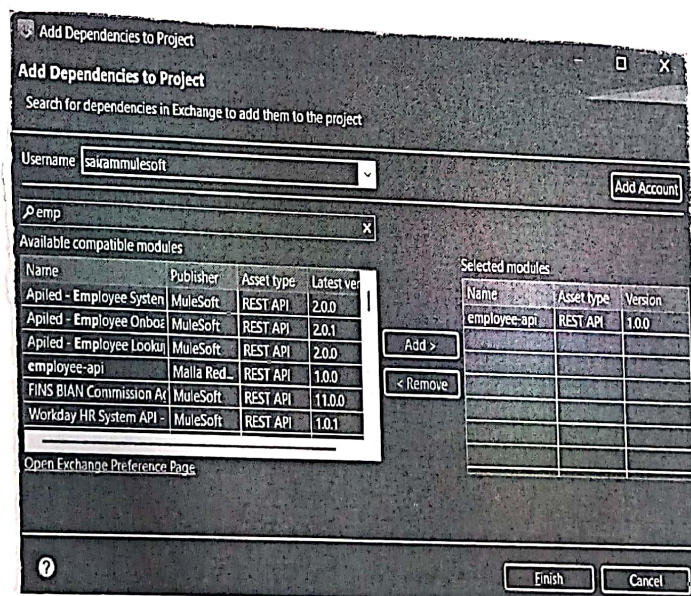
- Launch Anypoint Studio on your system
- Go to File → New → Mule project
 - Enter the Project Name: EmployeeAPIConsumer & click Finish
- Click import a published API, then click + button.



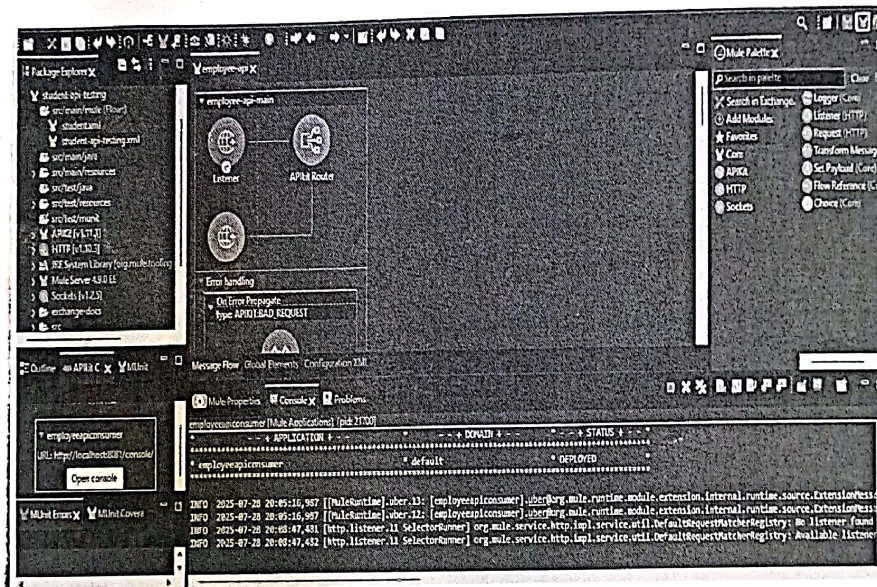
- In the Add dependencies to project dialog
- your Anypoint Username should be visible in the Username field
 - If not click Add Account and log in using your Anypoint platform credentials



- In the search bar, type the name eg employee API
- Select API & click Add > to move right side.

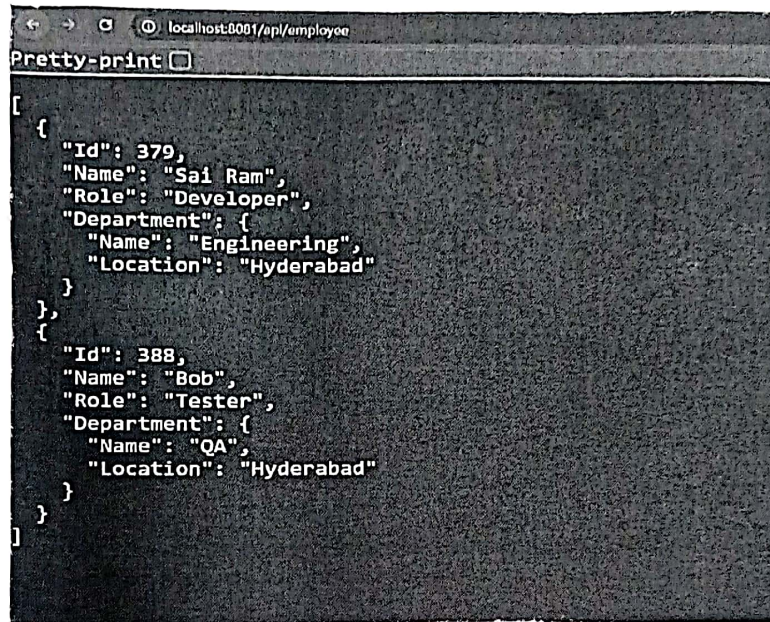


- click Finish & Again click Finish again
- your Mule project will now include:
 - The RAML files under src/main/resources/api
 - The Scaffolded flow for each method and resource defined in the RAML
 - pre-built router using APIKit to handle /employeepath
- open the generated flow eg: employee-api.xml



- In the employee-api.xml file you will auto generated flows like:
 - get /employee:application/json

- Don't change anything & save it
- Wait until get the message deployed
- Now test your application by opening browser & open this URL `http://localhost:8081/api/employee`
- You will get output like this.



```
[
  {
    "Id": 379,
    "Name": "Sai Ram",
    "Role": "Developer",
    "Department": {
      "Name": "Engineering",
      "Location": "Hyderabad"
    }
  },
  {
    "Id": 388,
    "Name": "Bob",
    "Role": "Tester",
    "Department": {
      "Name": "QA",
      "Location": "Hyderabad"
    }
  }
]
```


Week-6

Create a Mule project to create a table in MySQL using the Database connector in Anypoint Studio.

Solution:

In this experiment, we will create a Mule application that interacts with a MySQL database using Mulesoft's Database module. The objective is to define a flow that executes a Data Definition Language (DDL) command to create a new table inside a MySQL database.

Steps:

- First open MySQL Workbench:
 - In this select the connection if not create a local instance

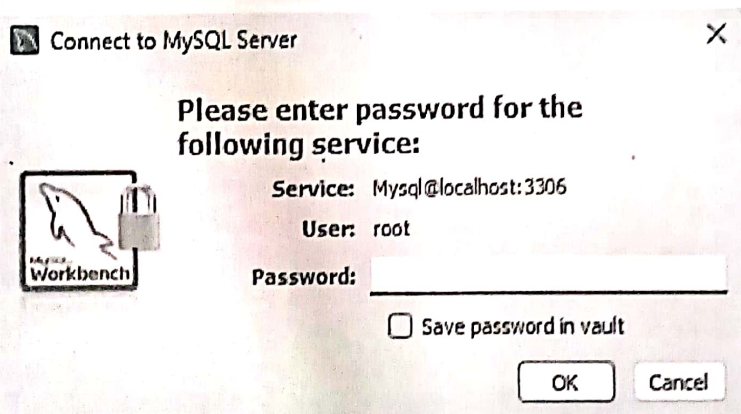
MySQL Connections ⊕ ⊖

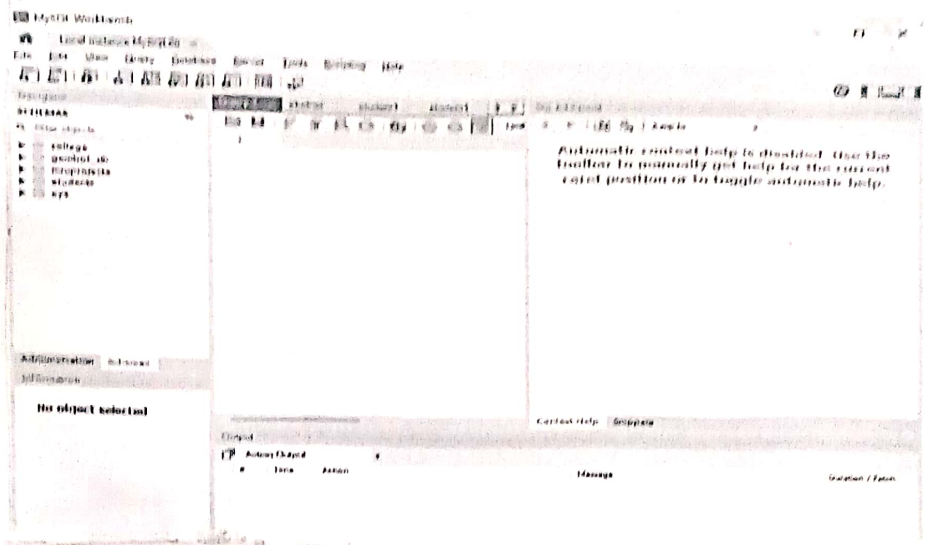
Local instance MySQL80

root

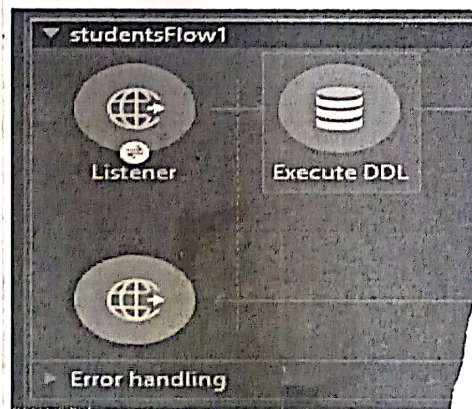
localhost:3306

→ connect to localhost by giving the password





- In this page create a database called students & click enter.
- open Anypoint studio & create a new mule project
 - go to file → New → Mule project
 - Name the project: sql project
 - click on finish
- In mule palette, expand the Database module. If it is not visible, click Add Modules and search for database.

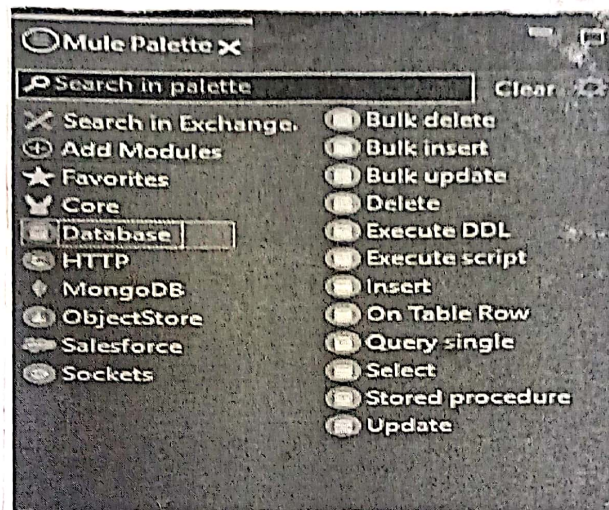


→ Drag an HTTP Listener component to the canvas

- Set the path as /create table
- configure the listener:

- Host: D.D.D.D
- port: 8081

→ Drag a Database - Execute PDL component next to the HTTP Listener.

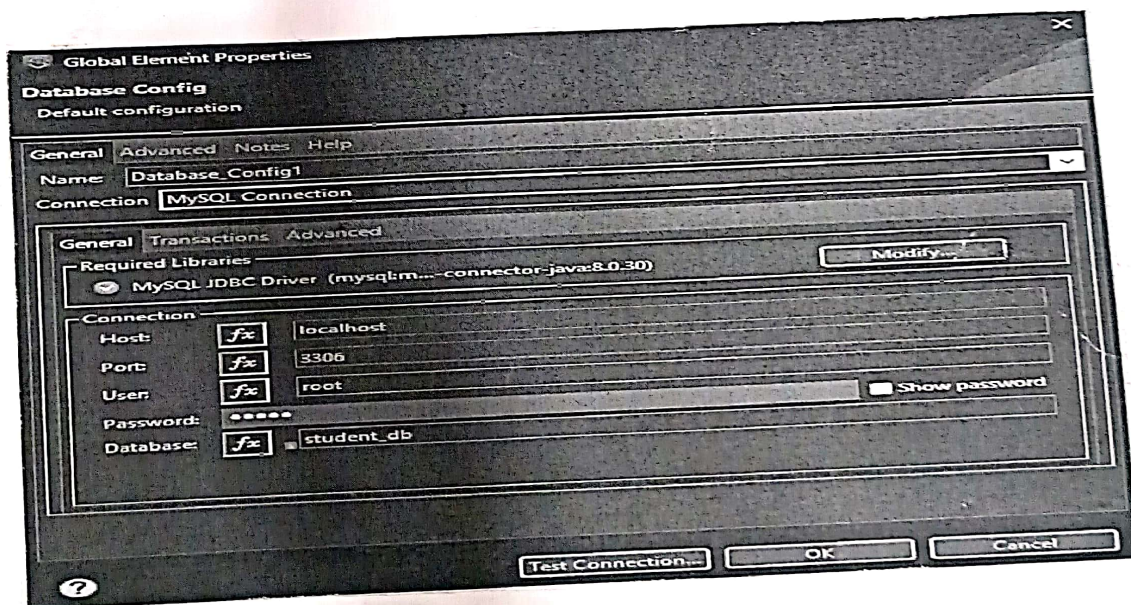


→ click on the database component and set up a global configuration:

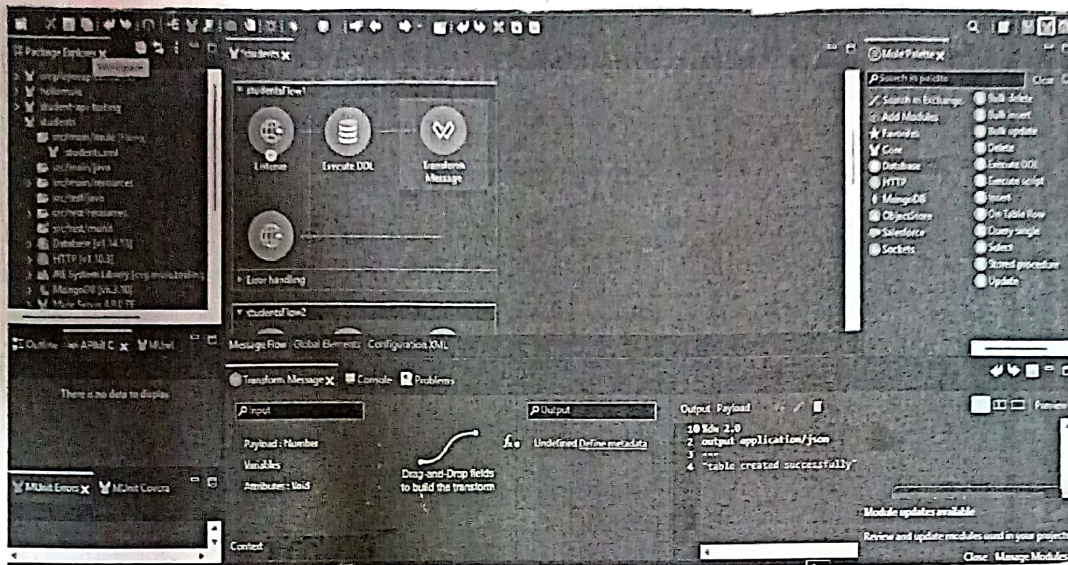
- Driver class name: com.mysql.cj.jdbc.Driver
- Host: localhost
- port: 3306
- user: root
- password: root
- Database: your-database-name
- Click Test connection to ensure it's successful



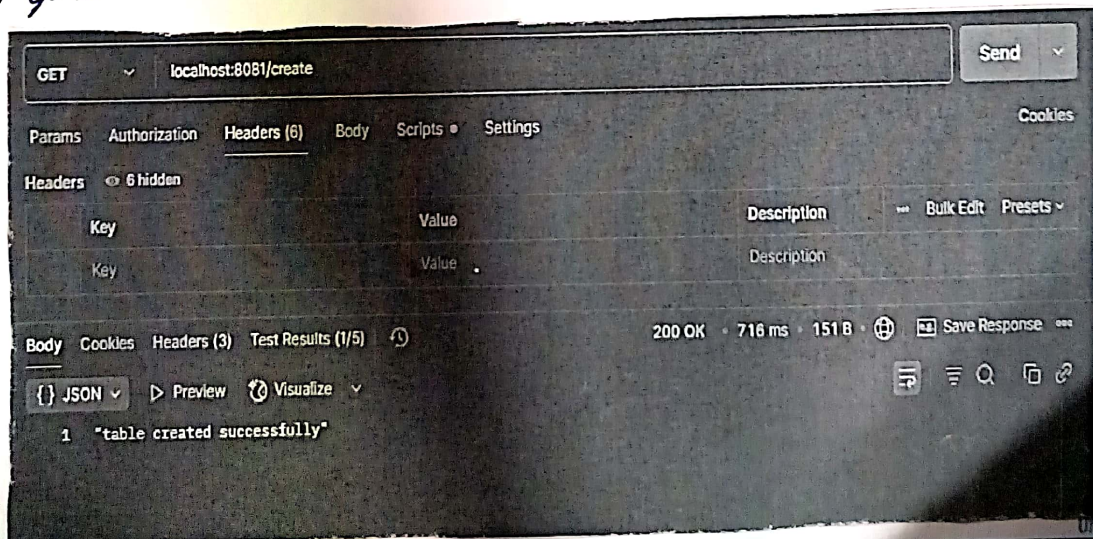
- In the Database configuration the connection select MySQL connection
- Near the Modify select it and select the Install Required Libraries.
- Click on Install Libraries and the Required Libraries are installed in the configuration
- Click on Test connection and click ok



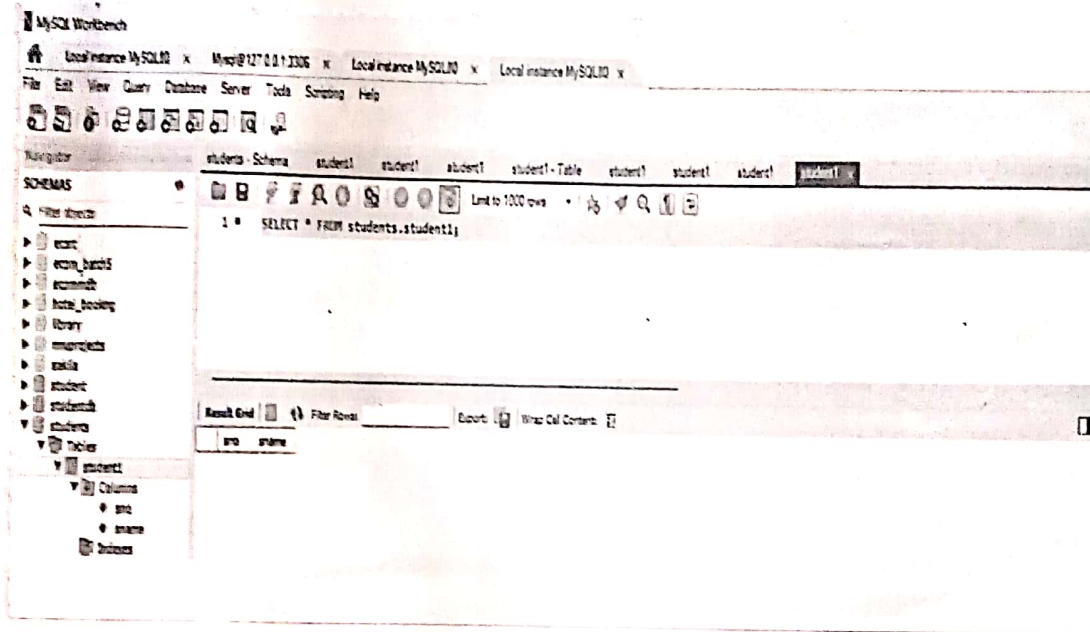
- After connection is successful click ok
- In the SQL query section of execute PDL component, write
create table student1 (sno varchar(20), sname varchar(20))
- Add a transform message and set the code to 1/son
and give a message for table created successfully.



- save the project and click Run
- open your browser & hit <http://localhost:8081/create>



- In the MySQL Workbench we can see that table is created.



EXPERIMENT-7

Create a mule project to insert and retrieve records from the MySQL table student2 using the database module.

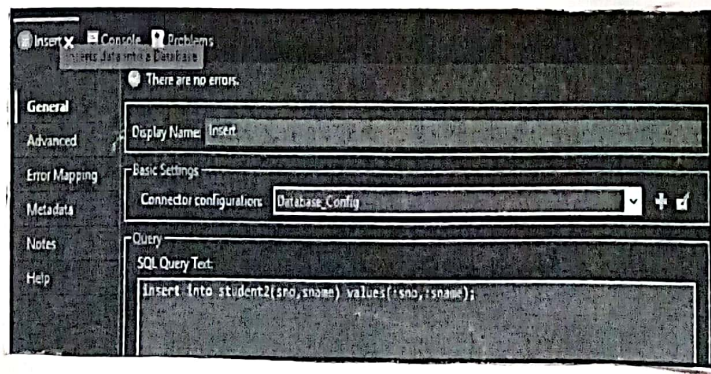
Solution:

In this experiment, you will build two flows:

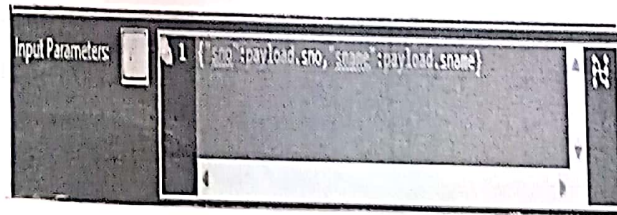
1. One to insert student records into the database
2. Another to retrieve and display those records as JSON

Steps:

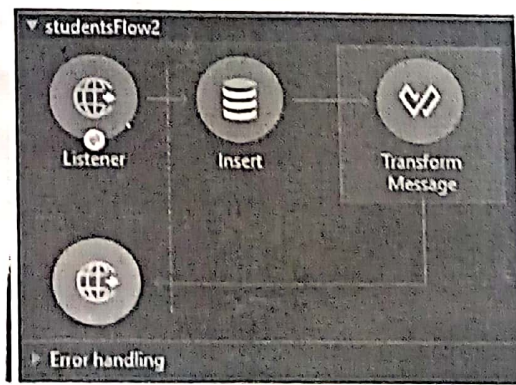
- continue the same project which was created in 6
- Drag an HTTP Listener to the canvas for inserting data
- path: /insert
- use the same configuration as the previous flow
- Drag a database-insert component after the transform.



- Use the same global DB config from 6
- SQL Query:
 - Insert into student2(sno,sname) values (:sno, :sname);
- Add the input parameters.



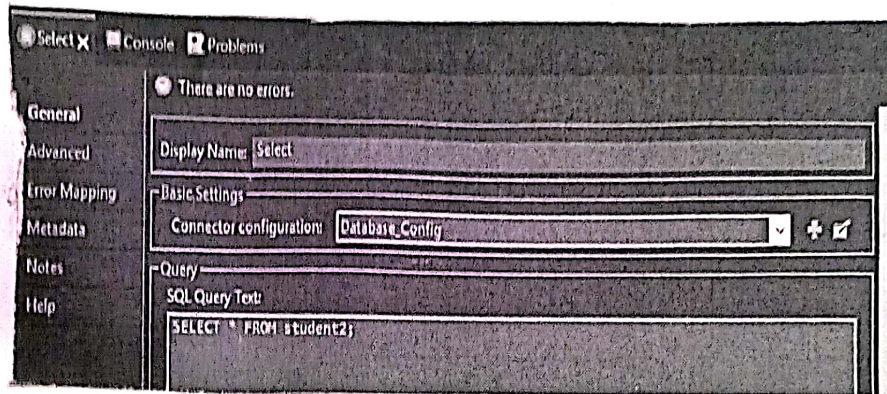
- Add a Transform Message component after the Listener:
 - Add the message "inserted data successfully".
 - The final flow will look like this:



- Next add another HTTP Listener for retrieval
 - path: /getstudents
 - config same as the previous flow
- Drag a Database-Select component after listener

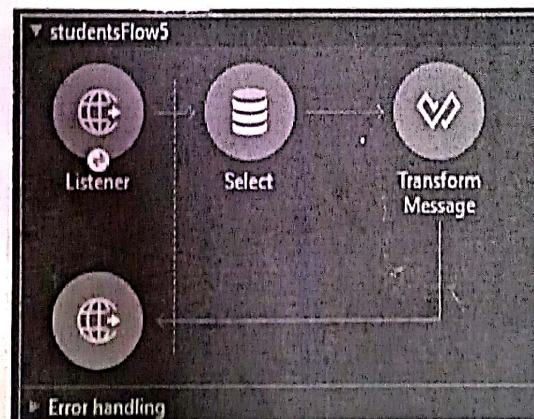
- SQL Query:

- Select * from student1;



- Add a Transform Message component to format the response as JSON

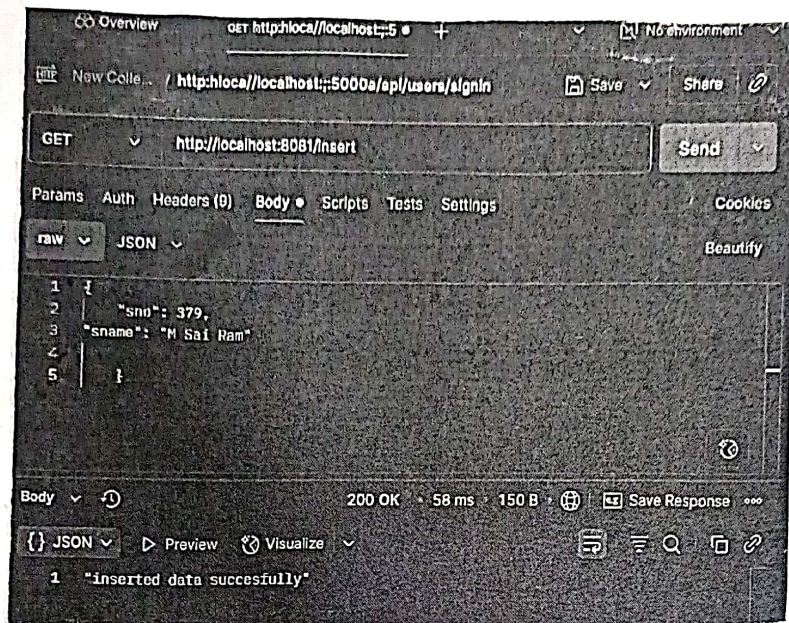
- The final will look like this.



- Next save the project & run it

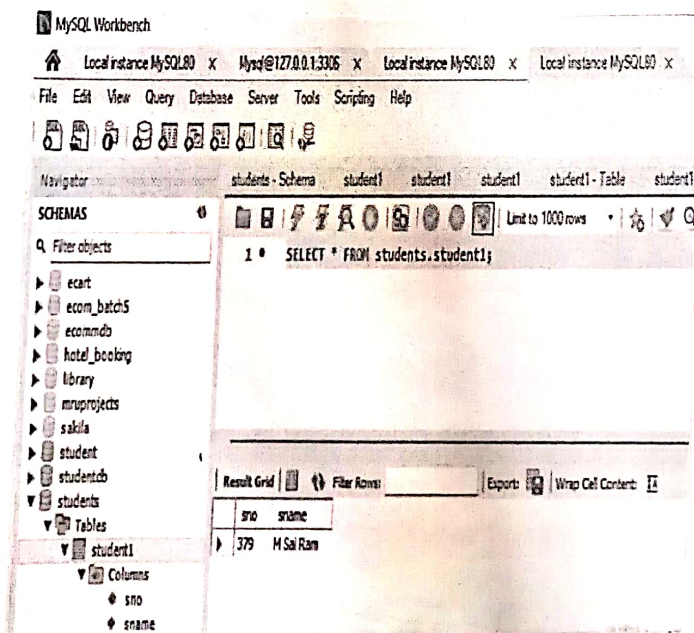
- Open postman & try to insert the data you should get output like this!

• open postman & hit `http://localhost:8081/insert`

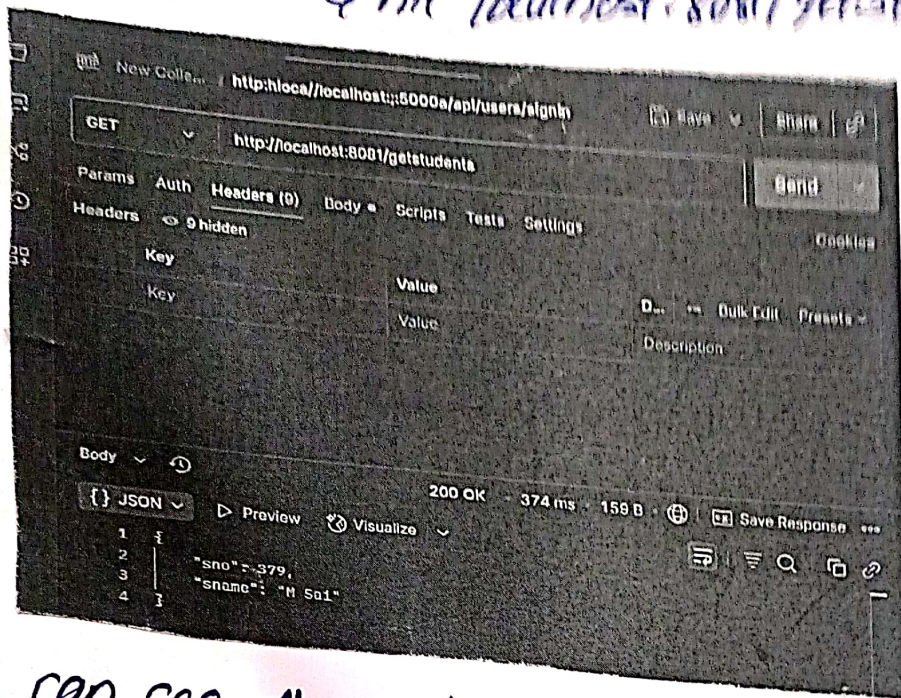


• "Inserted data successfully."

• In MySQL Workbench you can see that the data is inserted into table.



- open postman & hit localhost:8081/getstudents



- We can see the output in the postman

EXPERIMENT-8

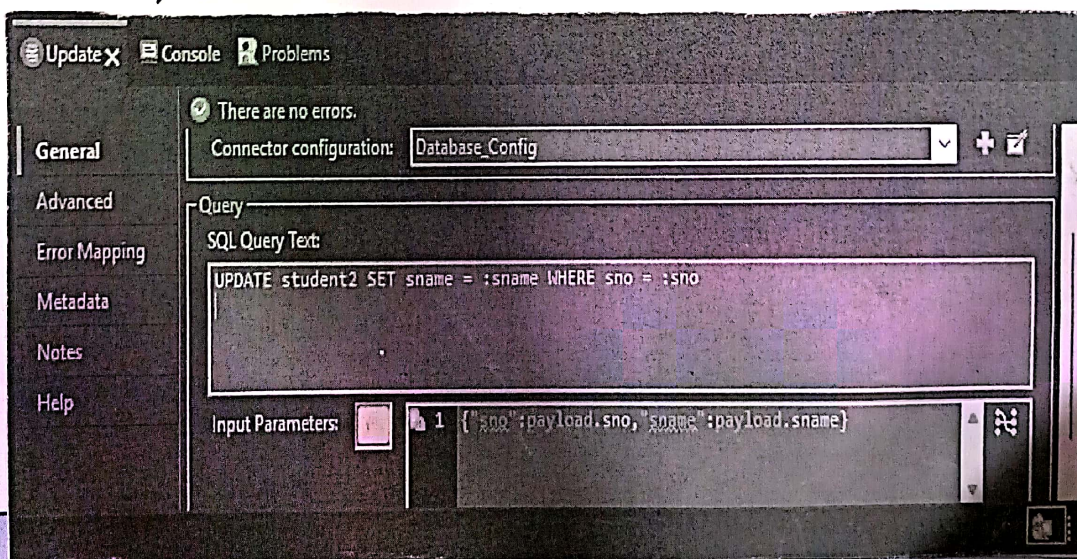
create a Mule project to update & delete records from the MySQL table using Database operations.

Solution:

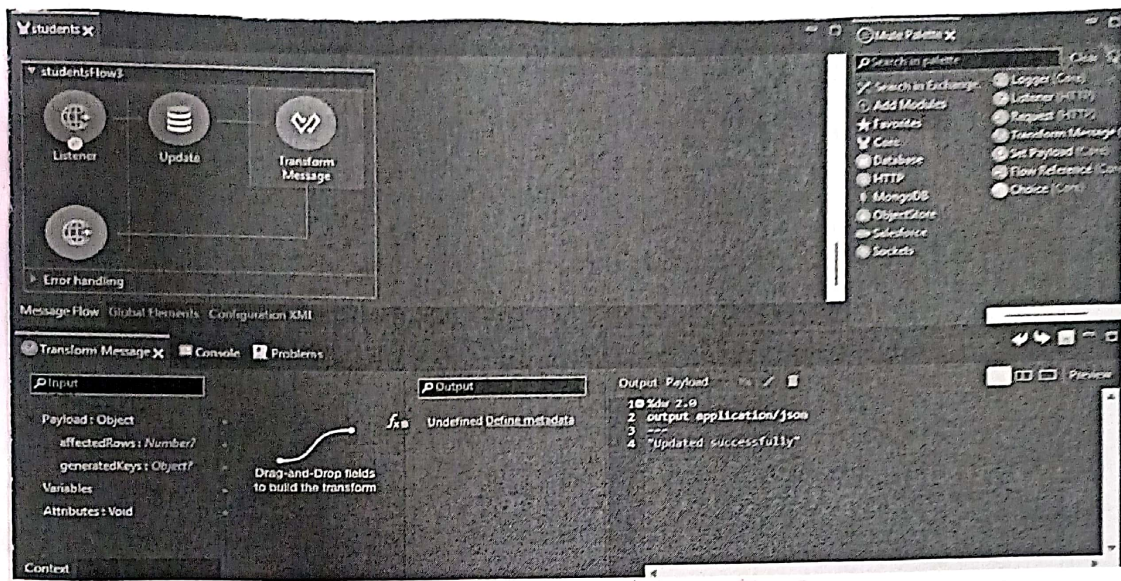
This experiment enhances the database integration by adding UPDATE and DELETE functionality using Mule's database connector.

Steps:

- Continue in your existing Mule project
- For update, drag an HTTP Listener:
 - path: /updateStudent
 - Keep the same config as the previous flows.
- Drag a database -update component:
 - For the config keep the database global config
 - For the SQL query:
 - Update student1 set sname = :sname where sno=:sno
 - Keep the input parameters;



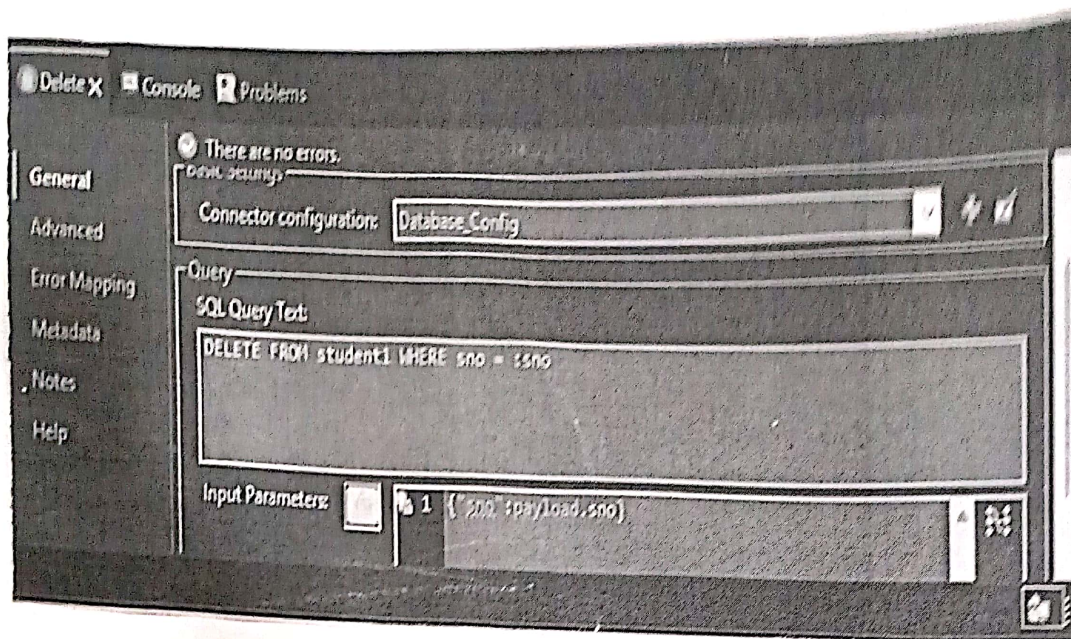
- Drag a transform message component with the json message "Updated successfully"
- The final flow will be:



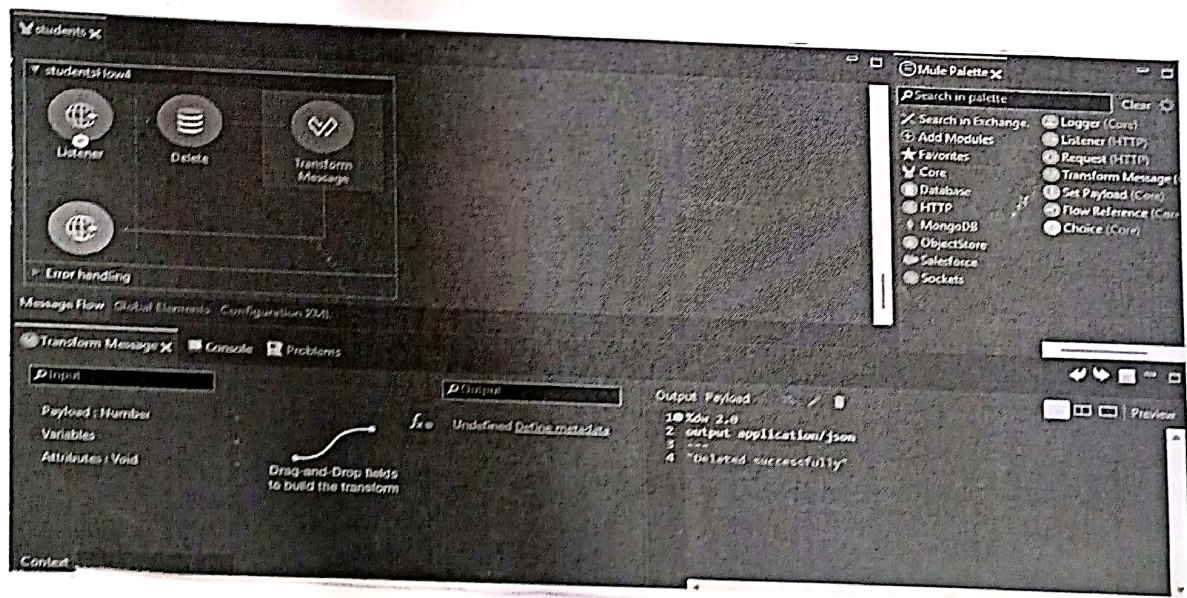
- Save the flow
- For Delete, drag another HTTP Listener:
 - path: /delete student
 - keep the same config as the previous flows.
- Drag a database - Delete component:
 - For the database config as the previous flows
 - keep the sql query like this:

Delete from student where sno = :sno
 - keep the input parameters:

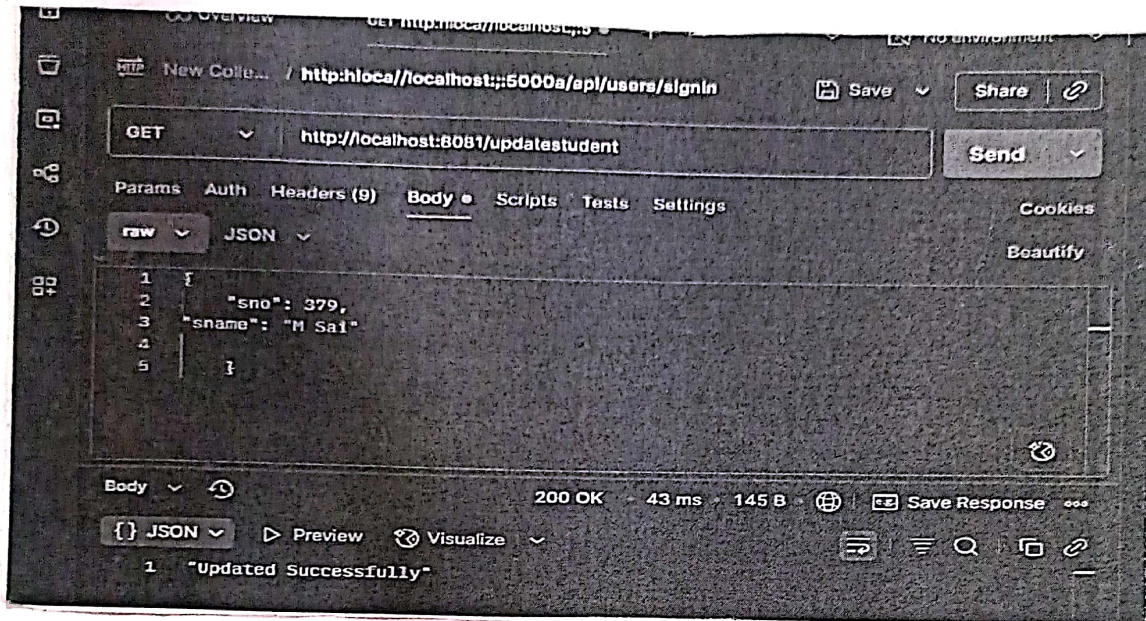




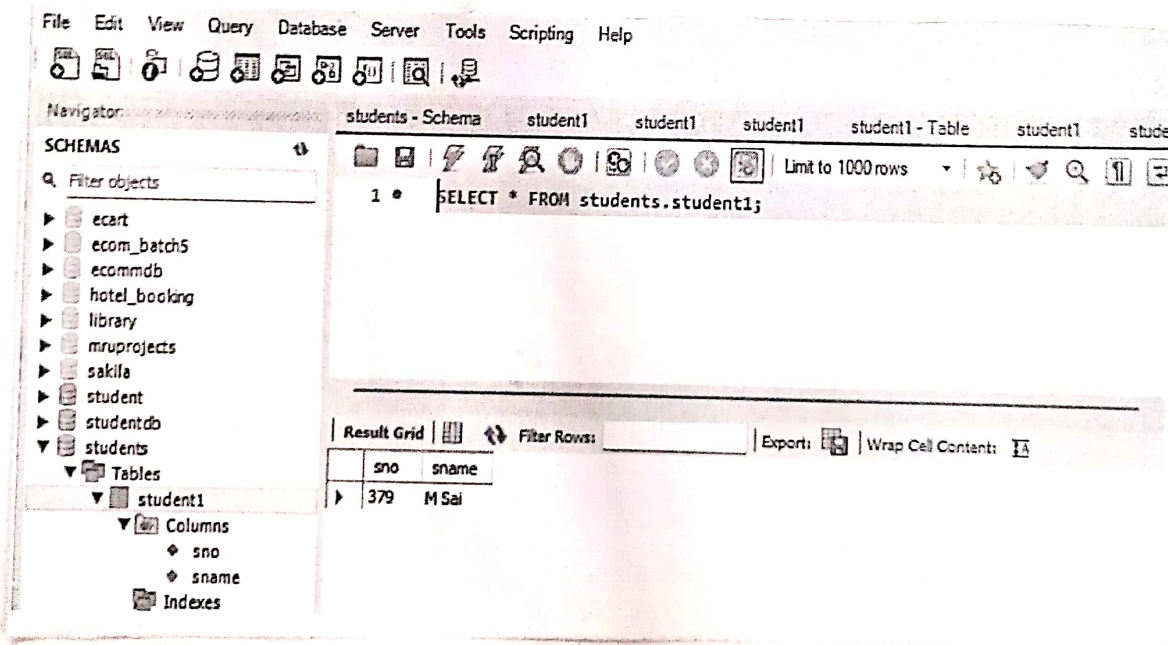
- Drag the transform message component & keep the message as data deleted successfully.
- The final flow will look like this:



- save the file and run the project
- go to postman & check if both flows are executing correctly
- for update flow the output will be:



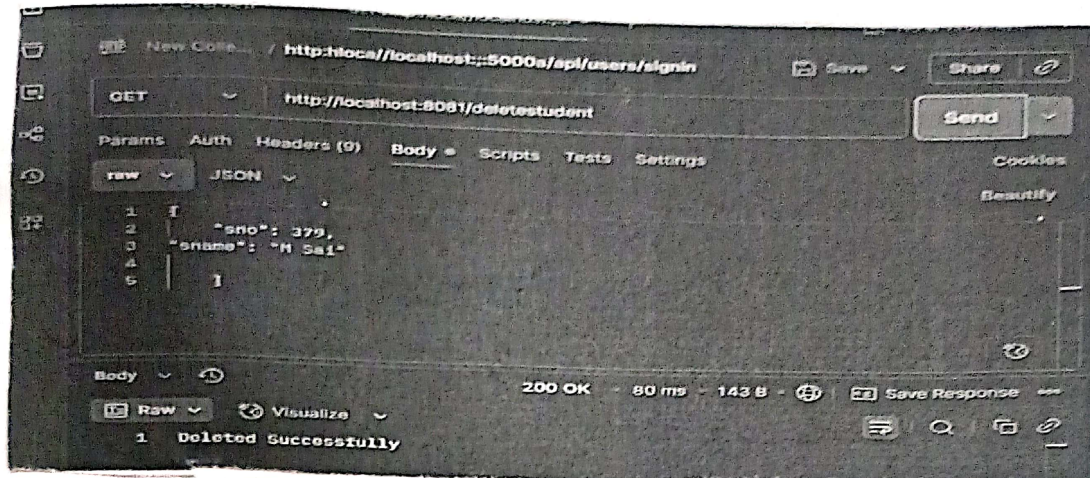
- In the postman hit `http://localhost:8081/updatestudent` the data is updated.



- In the MySQL Workbench we can see the data is updated.

for delete the output will be like this:

- In postman `http://localhost:8081/deletestudent`



• We can observe the data is successfully deleted.

• In the Workbench the data is deleted.

